

Tecnatura Superior en Desarrollo de Software

Guía de Implementación — CU-CLI-002: Actualizar Cliente

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

Contexto

Ya implementaste **CU-CLI-001 — Alta de Cliente**. Conocés el patrón completo: el UseCase recibe un Request, delega la validación a los Value Objects, persiste la entidad y notifica el resultado a través del OutputPort.

Ahora vas a implementar **CU-CLI-002 — Actualizar Cliente**. El objetivo es el mismo, pero el problema es diferente: el cliente ya existe, y solo algunos de sus datos pueden cambiar.

Esta guía no te da el código. Te hace las preguntas correctas para que vos lo construyas. Leé cada sección, pensá la respuesta y luego escribí el código.

Paso 1 — Dominio: el método Update() en la entidad Customer

Abrí la entidad `Customer`. Ya tiene un método estático `Create()` que recibe todos los datos y construye un cliente nuevo. Para la actualización necesitás un método de instancia: `Update()`.

¿Qué campos recibe Update()?

El caso de uso establece que **Nombre, Apellido y Documento son inmutables** (RN-001): una vez registrados, no pueden cambiar. Los campos editables son: Email, Teléfono, Dirección y Fecha de nacimiento.

Entonces, ¿qué parámetros debe recibir `Update()` ? Pensalo así: si un dato no puede cambiar, no tiene sentido recibirlo como parámetro. Recibir algo que luego ignorás es una contradicción. Y si el setter de esa propiedad es `private`, el compilador ya hace imposible asignarla desde afuera de la clase. No necesitás validar la inmutabilidad con código extra — la arquitectura la garantiza.

Para pensar: ¿Qué diferencia hay entre que algo sea imposible por diseño y que algo esté prohibido por una validación en tiempo de ejecución? ¿Cuál de los dos es más robusto y por qué?

¿Por qué `Update()` no recibe el ID del cliente?

El ID identifica al objeto sobre el que ya estás parado. Cuando escribís `customer.Update(...)`, ya tenés en memoria la instancia del cliente que querés modificar. Pasarle su propio ID como parámetro sería redundante: la entidad ya sabe quién es.

El ID lo usa el UseCase para encontrar al cliente en el repositorio. Una vez que lo encontró y tiene la instancia en memoria, el ID ya cumplió su función de localización. La entidad no lo necesita para actualizarse.

¿Qué hace `Update()` internamente?

Mirá cómo está construido `Create()`: recibe primitivos y los convierte en Value Objects antes de asignarlos a las propiedades. Cada VO valida su propio valor al ser creado — si algo está mal, lanza una `DomainException` en ese momento. Eso es el patrón Fail Fast.

`Update()` sigue exactamente el mismo patrón para los cuatro campos editables. La entidad siempre debe estar en un estado válido: nunca guardés un valor crudo en una propiedad que usa VO.

¿Qué retorna `Update()`?

Nada. Retorna `void`. `Update()` es una operación que modifica el estado interno de la entidad. No produce un objeto nuevo — transforma el que ya existe.

Compará con `Create()`, que es `static` y retorna una instancia nueva de `Customer` porque está construyendo algo de la nada. `Update()` trabaja sobre algo que ya existe en memoria. El resultado de la operación se refleja en las propiedades del objeto, no en un valor de retorno.

Tarea — Paso 1: Agregá el método `Update()` en `Customer.cs` con los cuatro parámetros editables. Convertí cada uno a su Value Object correspondiente antes de asignarlo a la propiedad. Si algún VO lanza una excepción, dejá que se propague: no la capturas dentro de la entidad.

Paso 2 — Dominio: extender `ICustomerRepository`

Abrí `ICustomerRepository`. Actualmente tiene dos métodos: uno para buscar por documento y otro para agregar un cliente. La actualización requiere operaciones nuevas. Pensá qué necesita el UseCase para poder hacer su trabajo y decidí qué métodos agregar.

¿Cómo se localiza al cliente que se va a modificar?

En CU-CLI-001 buscabas por documento para verificar duplicados. En CU-CLI-002 el sistema identifica al cliente por su ID. El repositorio necesita un método que encuentre un cliente por ID. Si no existe, ¿qué debe retornar? Mirá cómo lo resolvió el método por documento.

¿Cómo se verifica que el nuevo email no pertenece a otro cliente?

La regla RN-003 dice que el email debe ser único. Pero existe un caso especial: si el usuario guarda sin cambiar el email, el sistema encontraría ese email ya registrado en el propio cliente que está editando y lo rechazaría por duplicado cuando en realidad no lo es.

La solución es buscar en el repositorio quién tiene ese email. Si nadie lo tiene, está libre. Si alguien lo tiene, el UseCase puede comparar el ID de esa persona con el ID del cliente que está modificando. Si son el mismo, no hay conflicto. Si son distintos, el email pertenece a otra persona — eso sí es un duplicado.

Para hacer esa búsqueda, el repositorio necesita un método que encuentre un cliente por su email. ¿Qué retorna si ninguno tiene ese email?

¿Cómo se persiste la actualización?

Agregar un cliente nuevo usa `AddAsync`. Modificar uno existente es una operación distinta: el repositorio necesita un método que reciba la entidad con los datos ya modificados y la sincronice con el almacenamiento persistente. ¿Qué parámetro recibe ese método? ¿Qué retorna?

Tarea — Paso 2: Agregá los tres métodos nuevos a `ICustomerRepository` y `y`. Recordá que es una interfaz en la capa Domain: solo declara el contrato. La implementación concreta será responsabilidad de Infrastructure más adelante.

Para pensar: Cuando agregás métodos a una interfaz que ya tiene una clase implementadora, ¿el proyecto sigue compilando? ¿Qué error produce el compilador y qué te está indicando?

Paso 3 — Application: los DTOs de entrada y salida

UpdateCustomerRequest

El Request transporta los datos que llegan desde afuera del sistema hacia el UseCase. Seguí la misma convención que `CreateCustomerRequest`: es una `sealed class` con propiedades `required` e `init`.

¿Qué propiedades necesita? Pensá qué información necesita el UseCase para hacer su trabajo: el ID del cliente para encontrarlo, y los cuatro campos editables. Nombre, Apellido y Documento **no van en este DTO**. Si no llegan, es imposible usarlos para modificar la entidad, aunque alguien lo intentara. La ausencia del dato es, en sí misma, una barrera de seguridad.

UpdateCustomerResponse

El Response es lo que el UseCase entrega al OutputPort cuando la operación fue exitosa. Podrías pensar: los datos nuevos ya los tiene el front-end porque los ingresó el usuario, ¿para qué los devuelvo? ¿No alcanza con retornar el ID del cliente o un simple `bool` ?

No. Y hay razones de diseño importantes:

- **El sistema puede normalizar los datos.** Un Value Object puede transformar el valor recibido: por ejemplo, convertir el email a minúsculas, recortar espacios, o ajustar un formato. El valor que confirma el sistema puede ser diferente al que escribió el usuario. El front-end debe mostrar el valor real persistido, no la entrada cruda del usuario.
- **El Response expresa el contrato de lo que ocurrió.** Un `bool` no dice nada sobre qué cliente fue afectado ni en qué estado quedó. El Response es explícito: el ID confirma quién fue modificado, los valores confirman en qué estado quedó el registro.
- **El consumidor del OutputPort no tiene acceso directo a la entidad.** En una arquitectura desacoplada, quien recibe la notificación solo puede confiar en lo que el Response le provee. No puede ir a buscar los datos al repositorio por su cuenta.

Tarea — Paso 3: Creá `UpdateCustomerRequest` y `UpdateCustomerResponse` en la carpeta `Customers/DTOs`. Pensá qué propiedades necesita cada uno y por qué.

Paso 4 — Application: los puertos de entrada y salida

IUpdateCustomerInputPort

Seguí el mismo patrón de `ICreateCustomerInputPort`. Es la interfaz que expone el caso de uso hacia el mundo exterior. Define un único método que recibe el Request y retorna una `Task`.

IUpdateCustomerOutputPort

Este es el contrato que el UseCase usa para notificar resultados. Pensá en todos los caminos posibles que puede tomar la ejecución y definí un método para cada uno.

Caminos posibles en CU-CLI-002:

- El cliente con ese ID no existe en el sistema.
- El nuevo email ya le pertenece a otro cliente distinto (RN-003).
- Un Value Object rechazó un dato: email inválido, fecha futura, menor de edad.
- Todo salió bien: la actualización fue exitosa.

Para pensar: En CU-CLI-001 el canal de duplicado se refería al documento. En CU-CLI-002, ¿qué campo puede estar duplicado? ¿Debería el nombre del método reflejarlo para que el código sea más expresivo? ¿Qué ventaja tiene un método con nombre claro frente a un método genérico que recibe un código de error?

Tarea — Paso 4: Creá los dos puertos en la carpeta `Customers/Ports`. Usá `ICreateCustomerOutputPort` como referencia de estructura, pero no lo copies sin pensar: algunos canales son distintos o tienen una firma diferente.

Paso 5 — Application: el UseCase UpdateCustomerUseCase

El UseCase es el corazón de la implementación. Orquesta todo: recibe el Request, aplica las reglas de negocio, coordina con el repositorio y notifica el resultado a través del OutputPort. No contiene lógica de presentación ni de persistencia concreta.

A continuación están los pasos que debe seguir la ejecución, en orden. Para cada uno, pensá qué condición evaluás, qué método del OutputPort llamás si algo falla, y qué hacés si todo está bien antes de continuar.

Paso A — Localizar al cliente

Lo primero es encontrar al cliente que se quiere modificar. Usás el ID que llegó en el Request para buscarlo en el repositorio. ¿Qué hacés si no existe? ¿A qué método del OutputPort le avisás? ¿Podés seguir ejecutando después de eso?

Paso B — Verificar la unicidad del email

El nuevo email no puede pertenecer a otro cliente. Buscás en el repositorio quién tiene ese email actualmente.

Ojo: si encontrás un cliente con ese email y ese cliente es el mismo que estás editando, no hay conflicto. ¿Cómo distinguís si el email pertenece al mismo cliente o a otro? ¿Qué datos comparás? ¿A qué método del OutputPort llamás si hay conflicto real?

Paso C — Ejecutar la actualización en la entidad

Hasta acá el cliente existe y el email está libre. Ahora le pedís a la entidad que se actualice con los nuevos datos.

El método `Update()` puede lanzar una `DomainException` si algún Value Object rechaza el valor recibido: por ejemplo, un email con formato inválido o una fecha de nacimiento que indica minoría de edad. ¿Dónde capturás esa excepción? ¿Qué método del OutputPort usás para notificarla?

Paso D — Persistir los cambios

Si la entidad se actualizó sin errores, le pedís al repositorio que guarde los cambios. ¿Qué método del repositorio usás para eso?

Paso E — Notificar el éxito

Todo salió bien. Construís el Response y llamás al método de éxito del OutputPort. ¿De dónde sacás los valores para el Response? Recordá que los Value Objects pueden normalizar los datos. ¿Es mejor leerlos del Request o de la entidad ya actualizada?

Tarea — Paso 5: Implementá `UpdateCustomerUseCase` en la carpeta `Customers/UseCases`. Inyectá el repositorio y el `OutputPort` por constructor, igual que en `CreateCustomerUseCase`. Seguí los pasos A a E en orden. Cada vez que notificás un error por el `OutputPort`, la ejecución debe detenerse — pensá cómo lograrlo.

Para pensar: ¿Por qué el UseCase debe llamar a `return` inmediatamente después de notificar un error al `OutputPort`? ¿Qué podría ocurrir si la ejecución continuara? Pensá en un escenario concreto donde la falta del `return` produzca un comportamiento incorrecto.

Paso 6 — Application: registrar en `DependencyContainer`

Abrí `DependencyContainer.cs` en la capa Application. Ya tiene registrado el UseCase de alta. Necesitás registrar el nuevo UseCase con el mismo patrón.

¿Qué interfaz se registra como servicio? ¿Qué clase concreta la implementa? ¿Qué ciclo de vida corresponde y por qué es el mismo que el del UseCase de alta?

Tarea — Paso 6: Agregá la registración de `UpdateCustomerUseCase` dentro del método `AddCustomerUseCases()`. No olvides agregar los `using` necesarios en la parte superior del archivo.

Verificación final

Antes de dar por terminada la implementación, verificá cada punto. Si alguno no se cumple, revisá el paso correspondiente.

¿Qué verificar?	Capa
<code>Customer</code> tiene el método <code>Update()</code> con solo los 4 campos editables como parámetros	Domain
<code>Update()</code> convierte cada dato a su Value Object antes de asignarlo	Domain

¿Qué verificar?	Capa
<code>FirstName</code> , <code>LastName</code> y <code>Document</code> no aparecen en ningún parámetro de <code>Update()</code>	Domain
<code>ICustomerRepository</code> tiene un método para buscar por ID	Domain
<code>ICustomerRepository</code> tiene un método para buscar por email	Domain
<code>ICustomerRepository</code> tiene un método para persistir la actualización	Domain
<code>UpdateCustomerRequest</code> incluye el ID y los 4 campos editables. No incluye Nombre, Apellido ni Documento	Application
<code>UpdateCustomerResponse</code> incluye el ID y los valores confirmados por el sistema	Application
<code>IUpdateCustomerInputPort</code> declara un único método <code>ExecuteAsync</code>	Application
<code>IUpdateCustomerOutputPort</code> tiene un método para cada camino posible	Application
El UseCase localiza al cliente antes de cualquier otra operación	Application
La verificación de unicidad de email excluye correctamente al cliente que se está editando	Application
La <code>DomainException</code> que puede lanzar <code>Update()</code> está capturada y redirigida al <code>OutputPort</code>	Application
El Response se construye con los valores leídos de la entidad, no del Request	Application
<code>UpdateCustomerUseCase</code> está registrado en <code>DependencyContainer</code>	Application
El proyecto compila sin errores	—

Preguntas de reflexión

1. `Update()` no recibe el ID. `Create()` tampoco lo recibe, pero por una razón diferente en cada caso. ¿Cuál es la diferencia? ¿Por qué en ninguno de los dos tiene sentido recibir el ID como parámetro?
2. Si al verificar la unicidad del email encontrás un cliente con ese email pero no comparás su ID con el del cliente que estás editando, ¿qué escenario concreto falla incorrectamente? Describilo paso a paso.
3. El Response devuelve los valores confirmados por el sistema en lugar de un simple `bool`. ¿Qué problema concreto resuelve esto cuando un Value Object normaliza o transforma el dato que recibió?
4. Agregaste tres métodos nuevos a `ICustomerRepository`. Si Infrastructure todavía no los implementa, ¿el proyecto compila? ¿Qué tipo de error produce el compilador y en qué archivo lo señala? ¿Qué te está diciendo ese error?
5. La `DomainException` que puede lanzar `Update()` se captura en el UseCase y se redirige al OutputPort en lugar de propagarse. ¿Por qué no conviene dejarla propagar? ¿Quién debería manejarla si el UseCase no la captura? ¿Qué implicancias tiene eso para el diseño de las capas superiores?

Guía de Implementación — CU-CLI-002 Actualizar Cliente | Programación II | IES N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026